

TravelLuhh — Technical Documentation

Reference doc for understanding what exists, what each feature does, how data flows, and what's not built yet. Update when adding or removing features.

Table of Contents

- 1. [Tech Stack](#)
- 2. [Project Structure](#)
- 3. [Database Schema](#)
- 4. [Features](#)
- 5. [Authentication Flow](#)
- 6. [Trip Flow](#)
- 7. [Expense Flow](#)
- 8. [Budget System](#)
- 9. [API Endpoints Reference](#)
- 10. [Frontend Pages & Components](#)
- 11. [Known State / Tech Debt](#)
- 12. [Not Yet Built](#)

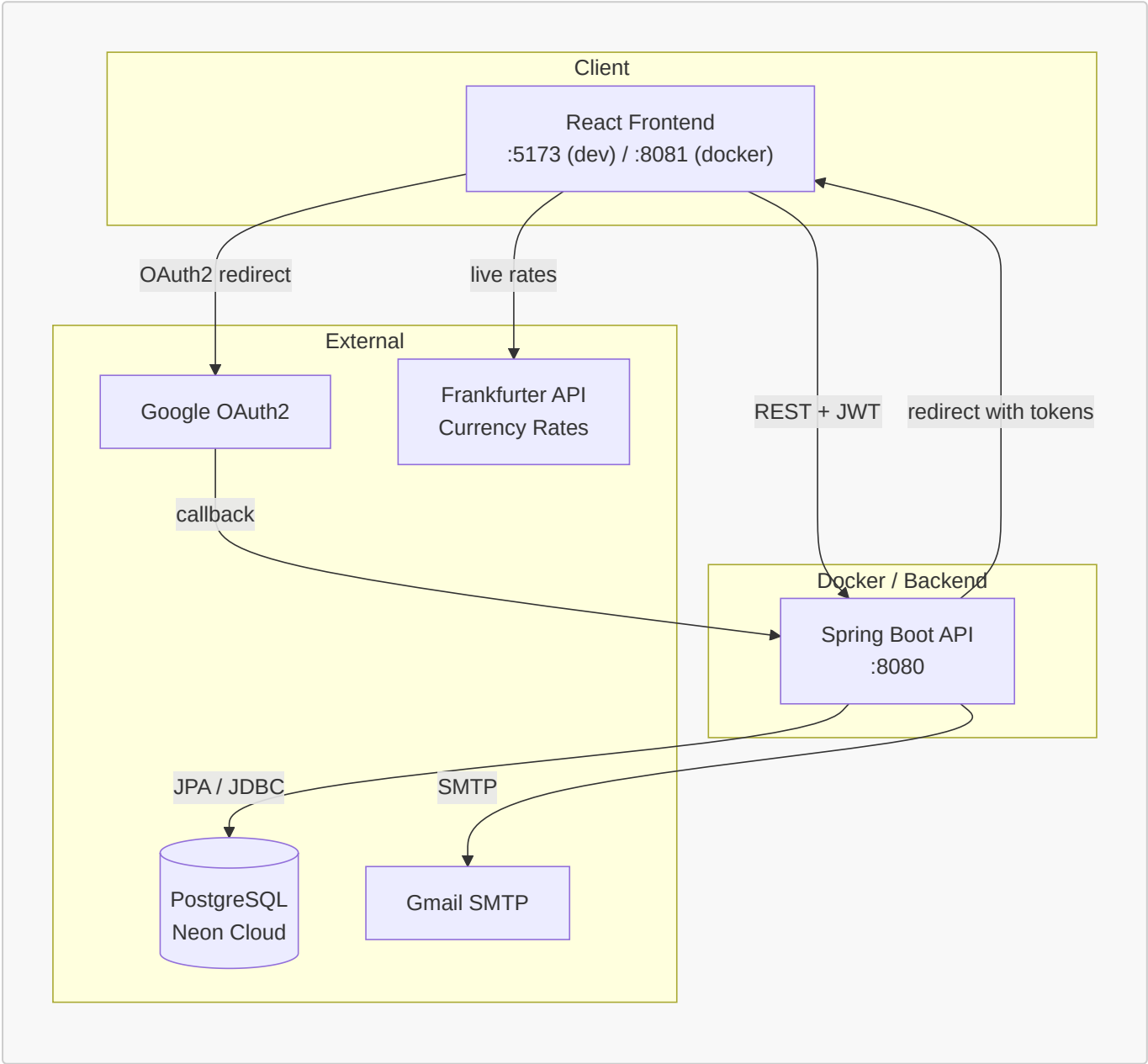
Tech Stack

Layer	Tech
Backend	Spring Boot 3.2.1, Java 17, Maven
Security	Spring Security + JWT (JJWT 0.12.3) + Google OAuth2
ORM	Spring Data JPA + Hibernate
Database	PostgreSQL (Neon cloud, ddl-auto=none)
Email	Gmail SMTP (async via JavaMailSender)
Containerisation	Docker + Docker Compose
Frontend	React 18 + TypeScript + Vite
Routing	React Router v6
HTTP Client	Axios (with auto token-refresh interceptor)
Server State	TanStack React Query
Forms	React Hook Form + Zod
UI	Tailwind CSS + shadcn/ui (Radix UI)
Charts	Recharts

Layer	Tech
Animations	Framer Motion
Icons	Lucide React
Currency API	Frankfurter (https://api.frankfurter.dev/v2)

Project Structure

High-Level System Architecture

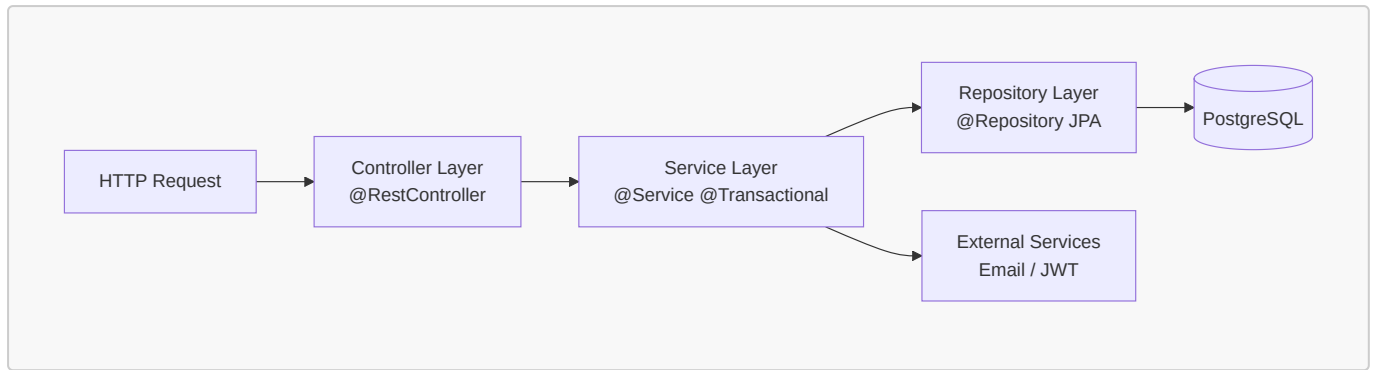


Directory Map

```
TravelLuhh/  
├── backend/  
│   └── src/main/java/com/travelluhh/  
│       ├── controller/           REST endpoints (@RestController)  
│       └── ...  
└── ...
```

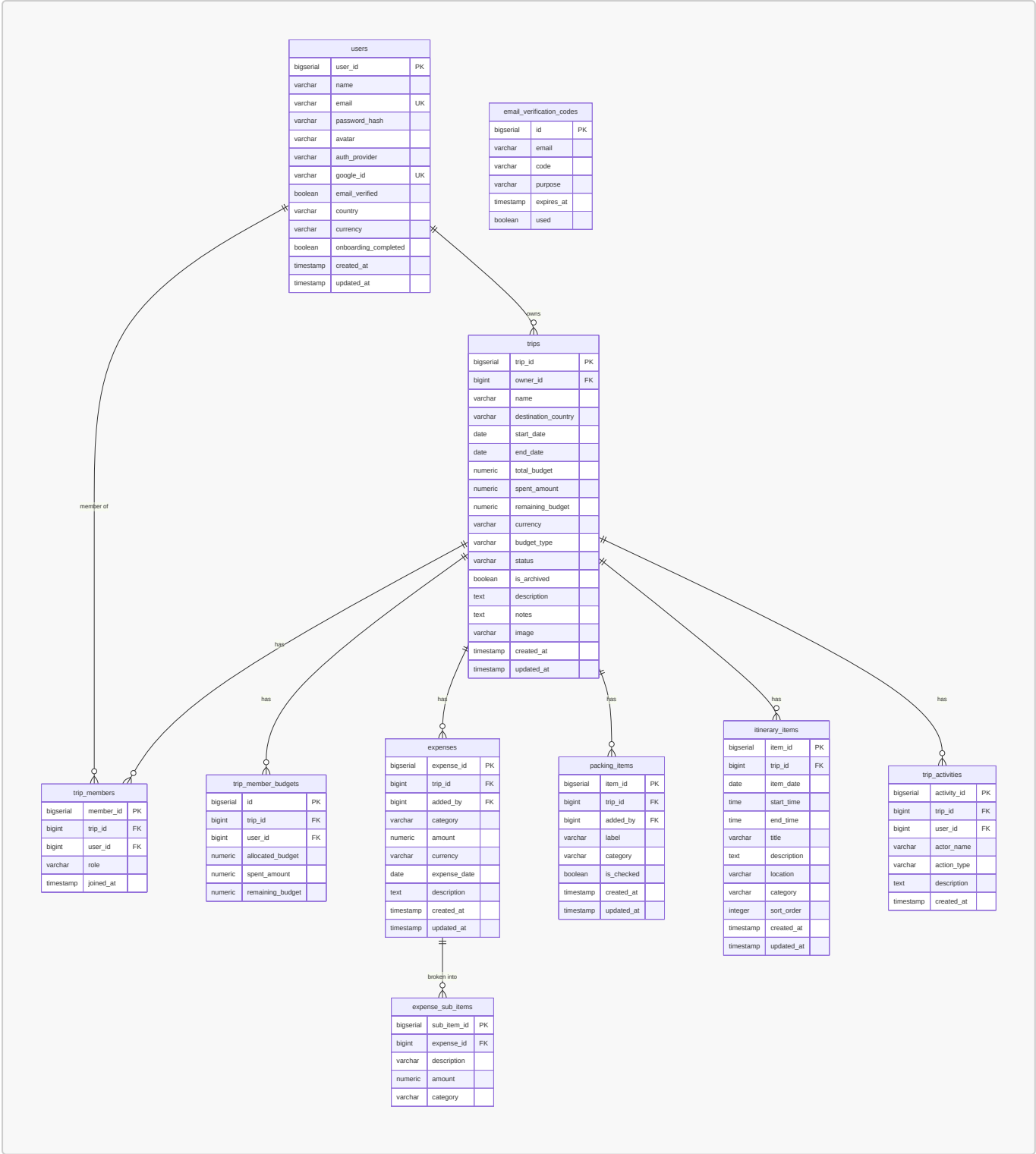
└─ service/	Business logic (@Service)
└─ repository/	JPA repos (@Repository)
└─ entity/	JPA entities → DB tables
└─ dto/	
└─ auth/	Login/signup request & response DTOs
└─ expense/	Expense request & response
└─ itinerary/	Itinerary item request & response
└─ packing/	Packing item request & response
└─ trip/	Trip request, response, settlement
└─ user/	Profile update, password change
└─ security/	JWT filter, OAuth2 handlers, SecurityConfig
└─ exception/	GlobalExceptionHandler + custom exceptions
└─ util/	JwtUtil
└─ src/main/resources/	
└─ application.yml	App config (DB, JWT, OAuth2, email, CORS)
└─ schema.sql	Full DB DDL (manual migrations)
└─ frontend/src/	
└─ pages/	Route-level components
└─ components/	
└─ activity/	ActivityFeed
└─ alerts/	BudgetAlerts
└─ auth/	GoogleLoginButton, ProtectedRoute,
TokenRefreshHandler	
└─ charts/	BentoCharts, BudgetCharts
└─ currency/	CurrencyConverter, TripCurrencyCard,
CurrencyCards	
└─ expenses/	ExpenseForm, ExpenseListWithTotal
└─ export/	ExportMenu
└─ itinerary/	TripItinerary
└─ members/	AddMemberDialog, MemberList,
BudgetAllocationPanel	
└─ notes/	EditableDestinationNotes
└─ onboarding/	OnboardingModal
└─ packing/	PackingList
└─ permissions/	PermissionGate
└─ portfolio/	TravelMap, TravelTimeline, YearInReview
└─ settlement/	SettlementSummary
└─ sharing/	TripSharing
└─ trips/	TripCard, TripFilters, TripForm
└─ services/	API call layer (authService, tripService,
etc.)	
└─ context/	AuthContext (global auth state)
└─ hooks/	useTrips, useExpenses, use-toast, use-
mobile	
└─ lib/	axios.ts, currency.ts, export.ts, utils.ts
└─ types/	TypeScript interfaces (index.ts)
└─ data/	countries.ts (static country + currency
data)	
└─ docker-compose.yml	
└─ README.md	

Backend Layer Architecture



Database Schema

Entity Relationship Diagram



Tables Overview

Table	Purpose
users	User accounts (local + Google OAuth)
email_verification_codes	OTP codes for signup + password reset
trips	Travel trips with budget info
trip_members	Links users to trips with roles
trip_member_budgets	Per-member budget allocation (separated mode)

Table	Purpose
<code>expenses</code>	Expenses per trip
<code>expense_sub_items</code>	Optional per-category breakdown inside a bundle expense
<code>packing_items</code>	Packing checklist items per trip
<code>trip_activities</code>	Activity log / feed per trip
<code>itinerary_items</code>	Day-by-day planned activities per trip
<code>destination_notes</code>	Trip notes — table exists, currently unused (notes stored in <code>localStorage</code>)
<code>budget_alerts</code>	Alert thresholds — table exists, currently unused (alerts computed client-side)

No soft delete. All tables use hard deletes. `is_deleted` columns have been fully dropped.

users

Column	Type	Notes
<code>user_id</code>	BIGSERIAL PK	
<code>name</code>	VARCHAR	
<code>email</code>	VARCHAR UNIQUE	
<code>password_hash</code>	VARCHAR	NULL for OAuth users
<code>avatar</code>	VARCHAR	Profile picture URL
<code>auth_provider</code>	VARCHAR	<code>local</code> or <code>google</code>
<code>google_id</code>	VARCHAR UNIQUE	NULL for local users
<code>email_verified</code>	BOOLEAN	Default false
<code>country</code>	VARCHAR	ISO-2 country code
<code>currency</code>	VARCHAR	Default preferred currency (ISO-4217)
<code>onboarding_completed</code>	BOOLEAN	False until profile setup done
<code>created_at / updated_at</code>	TIMESTAMP	

trips

Column	Type	Notes
<code>trip_id</code>	BIGSERIAL PK	
<code>owner_id</code>	BIGINT FK → users	
<code>name</code>	VARCHAR	

Column	Type	Notes
destination_country	VARCHAR	ISO-2
start_date / end_date	DATE	
total_budget	NUMERIC(12,2)	
spent_amount	NUMERIC(12,2)	Auto-updated by DB trigger
remaining_budget	NUMERIC(12,2)	Generated column: total_budget – spent_amount
currency	VARCHAR	Trip's display currency
budget_type	VARCHAR	solo, shared, or separated
status	VARCHAR	PLANNED, ONGOING, COMPLETED
is_archived	BOOLEAN	Hidden from active list when true
description / image / notes	TEXT/VARCHAR	
created_at / updated_at	TIMESTAMP	Auto-updated by trigger

DB Triggers:

- `trg_update_trip_spent` — recalculates `spent_amount` after any INSERT/UPDATE/DELETE on `expenses`
- `trg_update_member_budget_spent` — recalculates per-member spent in separated mode

expenses

Column	Type	Notes
expense_id	BIGSERIAL PK	
trip_id	BIGINT FK → trips	
added_by	BIGINT FK → users	NULL if user deleted
category	VARCHAR	accommodation, transportation, food, activities, shopping, bundle, other
amount	NUMERIC(12,2)	Amount in trip's currency
currency	VARCHAR	Trip currency
expense_date	DATE	
description	TEXT	
created_at / updated_at	TIMESTAMP	

bundle category: Package deal (e.g. flight + hotel). One total amount. Optional sub-items break it down per category via **expense_sub_items**.

expense_sub_items

Column	Type	Notes
sub_item_id	BIGSERIAL PK	
expense_id	BIGINT FK → expenses	Cascades on delete
description	VARCHAR	e.g. "Flight KUL-JED"
amount	NUMERIC(12,2)	
category	VARCHAR	Any category except bundle

packing_items

Column	Type	Notes
item_id	BIGSERIAL PK	
trip_id	BIGINT FK → trips	
added_by	BIGINT FK → users	
label	VARCHAR	Item name
category	VARCHAR	Optional grouping
is_checked	BOOLEAN	Default false

trip_activities

Column	Type	Notes
activity_id	BIGSERIAL PK	
trip_id	BIGINT FK → trips	
user_id	BIGINT FK → users	NULL if user deleted
actor_name	VARCHAR	Name snapshot at time of action
action_type	VARCHAR	expense_added , expense_deleted , trip_updated , etc.
description	TEXT	Human-readable log entry

itinerary_items

Column	Type	Notes
item_id	BIGSERIAL PK	

Column	Type	Notes
trip_id	BIGINT FK → trips	
item_date	DATE	Which day of the trip
start_time / end_time	TIME	Optional
title	VARCHAR	Required
description	TEXT	
location	VARCHAR	
category	VARCHAR	transport, accommodation, food, activity, sightseeing, shopping, other
sort_order	INTEGER	Default 0

trip_members

Column	Type	Notes
member_id	BIGSERIAL PK	
trip_id	BIGINT FK	
user_id	BIGINT FK	
role	VARCHAR	owner, editor, viewer
joined_at	TIMESTAMP	
UNIQUE	(trip_id, user_id)	One entry per user per trip

email_verification_codes

Column	Notes
email	Target email
code	6-digit OTP
purpose	signup or forgot_password
expires_at	10 minutes from creation
used	Flipped to true after use

Features

1. Authentication

- **Email signup** — 3-step: send OTP → verify OTP → complete with name + password

- **Email login** — Email + password, returns access + refresh token
- **Google OAuth2** — One-click login/signup; auto-creates account on first use
- **Password reset** — Send OTP → verify → set new password
- **Token refresh** — Axios interceptor silently refreshes expired access tokens
- **Onboarding modal** — After first login, prompts for country + currency

2. Trip Management

- Create, view, edit, delete trips
- Set destination country, dates, total budget, display currency
- Budget type: **solo** (one person), **shared** (group, one pot), **separated** (per-member allocation)
- Trip status lifecycle: **PLANNED** → **ONGOING** → **COMPLETED**
- Archive trips (hidden from active list, not deleted)
- Add description, cover image, and general notes

3. Expense Tracking

- Add expenses: category, amount, currency, date, description
- Track who added each expense (**added_by**)
- Delete expenses (creator or trip owner only)
- DB trigger auto-updates **trips.spent_amount** on any change
- Expenses displayed newest-first
- **Bundle/Package expenses**: one total cost for a package deal; sub-items break it down per category with an expandable row in the list

4. Budget System

Mode	Description
solo	One person, one budget
shared	Whole group shares one budget pool
separated	Each member has their own allocated amount in trip_member_budgets

- Real-time remaining = **total_budget** – **spent_amount** (DB generated column)
- Budget alert banners on trip detail (computed client-side at 75%, 90%, 100% thresholds)
- Alerts are dismissible per session

5. Trip Sharing & Members (UI Only — see Known State)

- **TripSharing** dialog shows: public access toggle, invite by email, collaborators list with roles
- Collaborator list populated from real **trip_members** data
- **Invite / remove / role-change actions are UI-only** — callbacks not wired to backend API

6. Analytics & Charts (Charts tab)

- Bento-card layout with: Top Category, Total Spent, Daily Target, Avg/Expense
- Budget Usage radial gauge
- Spending by Category pie + bar

- Daily spending trend (bar + cumulative area chart)
- Top 5 spending categories horizontal bar
- Budget vs Spent comparison bar
- Category breakdown detail cards (below bento section)

7. Destination Notes

- Per-trip notes stored in **localStorage** (key: `travelluhh_notes_<tripId>`)
- Categories: Local Tips, Food & Dining, Transportation, Safety, Emergency, Custom
- Destination quick info (timezone, language, emergency number, voltage, tipping) also stored in localStorage (key: `travelluhh_destinfo_<tripId>`)
- All notes persist across browser sessions but are device/browser-local only

8. Data Export

- Export trip as **CSV** (raw expense data)
- Export as **Print / Save PDF** (styled report via blob URL)
- Includes: trip summary, budget overview, full expense list

9. User Profile

- Update name, country, preferred currency
- Change password (validates current password first)
- Email is read-only

10. Packing Checklist

- Add/remove items per trip
- Check off items as packed; progress bar shows % complete
- Items grouped by category
- **Preset packs:** General, Beach, Hiking, Business, Umrah/Hajj — one-click to add a curated list

11. Itinerary / Day Planner

- Auto-generates a card per day for the trip duration
- Per item: title, category, start/end time, location, notes
- Items sorted by start time within each day
- Add/Edit/Delete via dialog

12. Activity Feed

- Chronological log of trip actions
- Tracked events: expense added, expense deleted, trip updated
- Shows actor name, description, relative timestamp ("2 hours ago")
- Logged asynchronously (wrapped in try-catch, does not block main transaction)

13. Split Bill / Settle Up

- Available for **shared** and **separated** trip types only
- Shows per-member: total paid, equal share, net balance

- Calculates who-owes-whom via greedy debt-simplification algorithm
- Equal-share model: total spend ÷ member count

14. Currency Converter (Quick Price Converter)

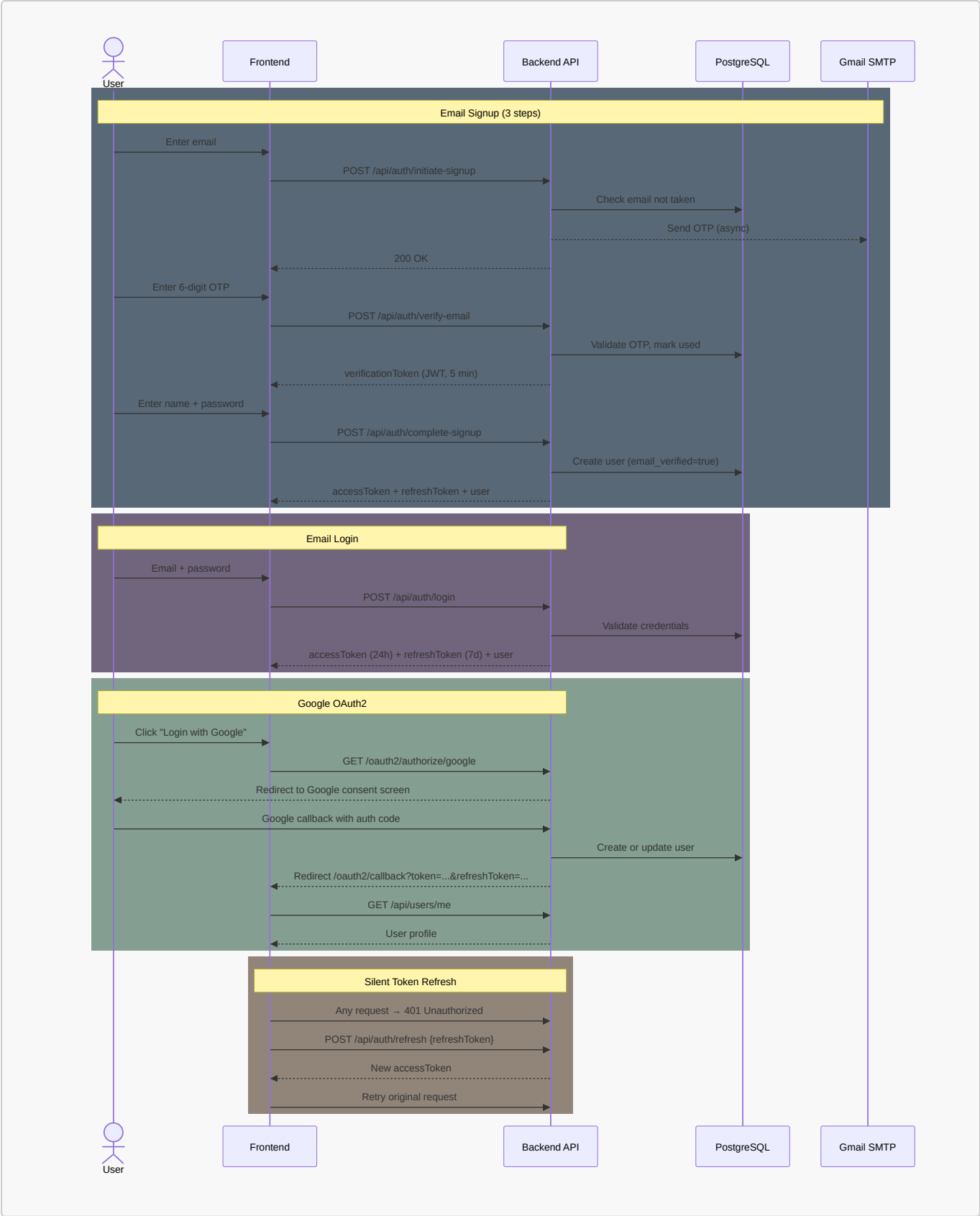
- Available on the Dashboard as "Quick Price Converter"
- Live rates from Frankfurter API, debounced 400 ms
- Swap button to flip from/to currencies
- Defaults to user's preferred currency

15. Portfolio

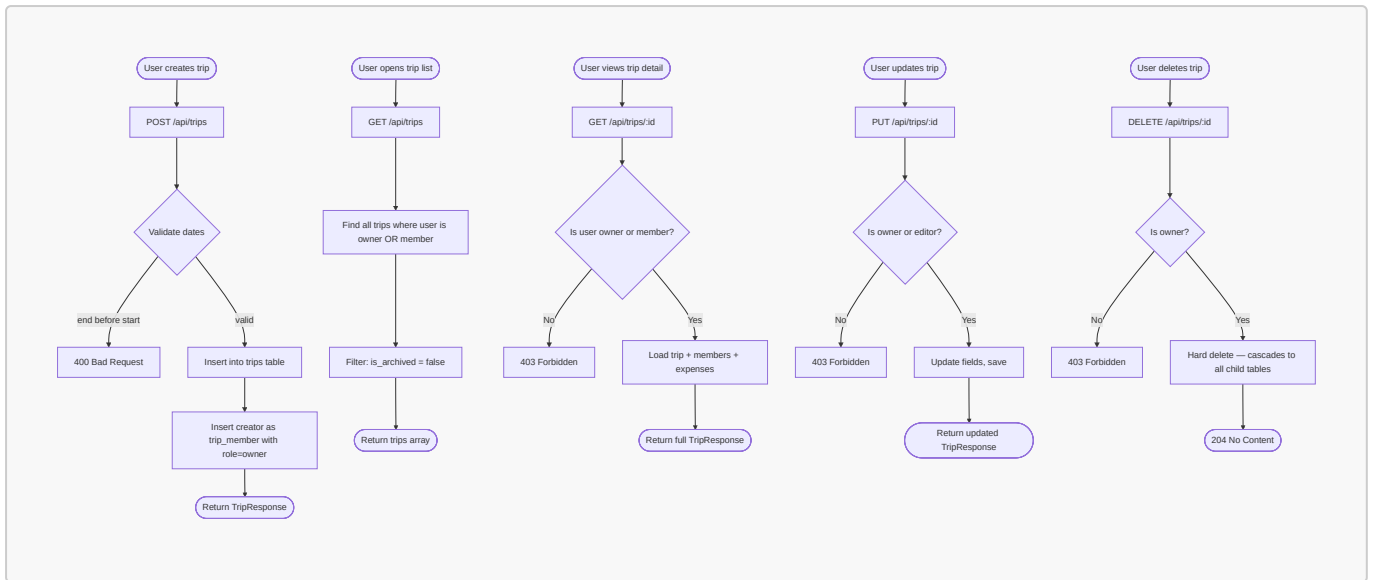
- **Year in Review** — Annual stats: trips, countries, total spend
 - **Travel Timeline** — Chronological history of all trips
 - **Travel Map** — Visual map of visited destinations
-

Authentication Flow

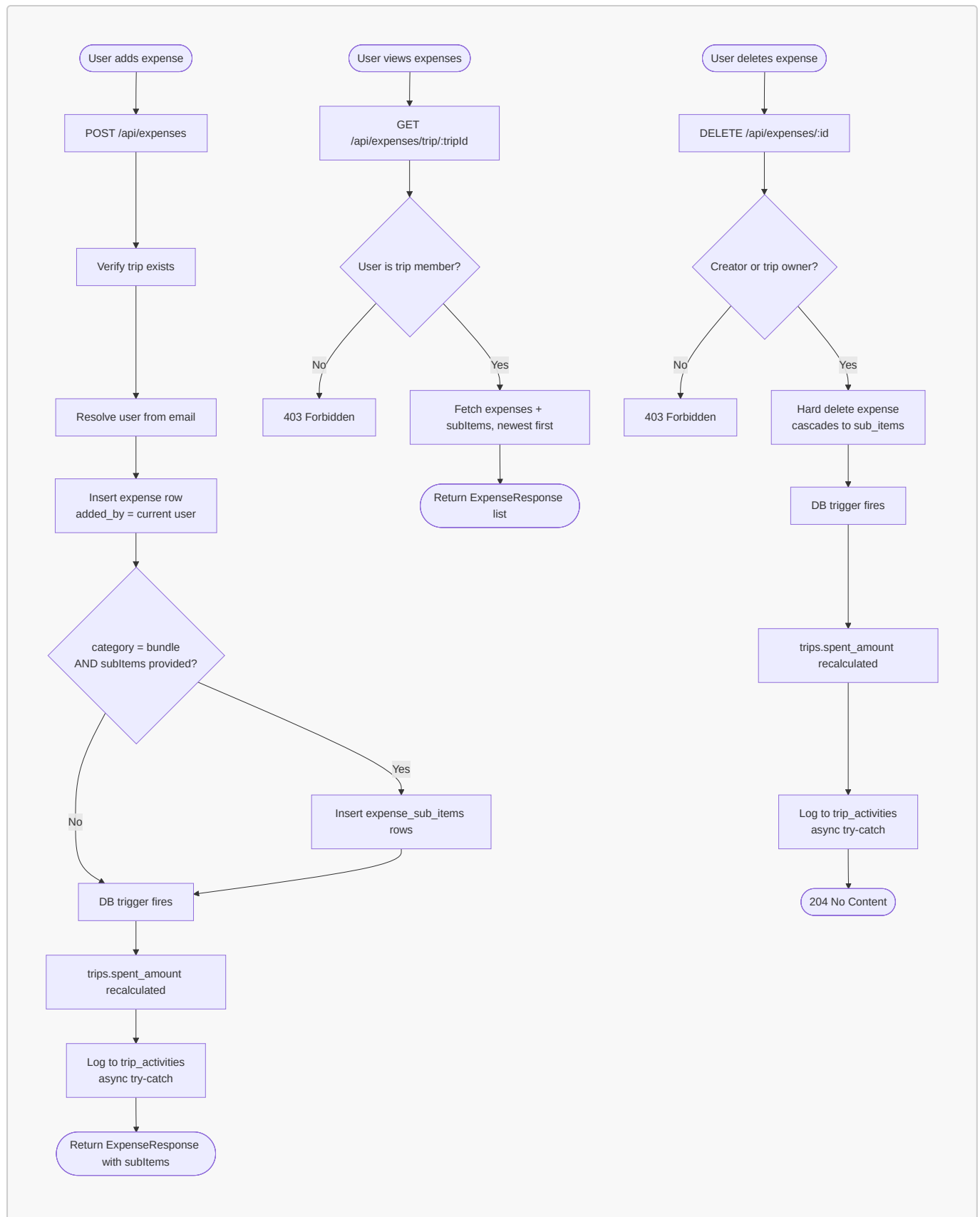
Sequence Diagram



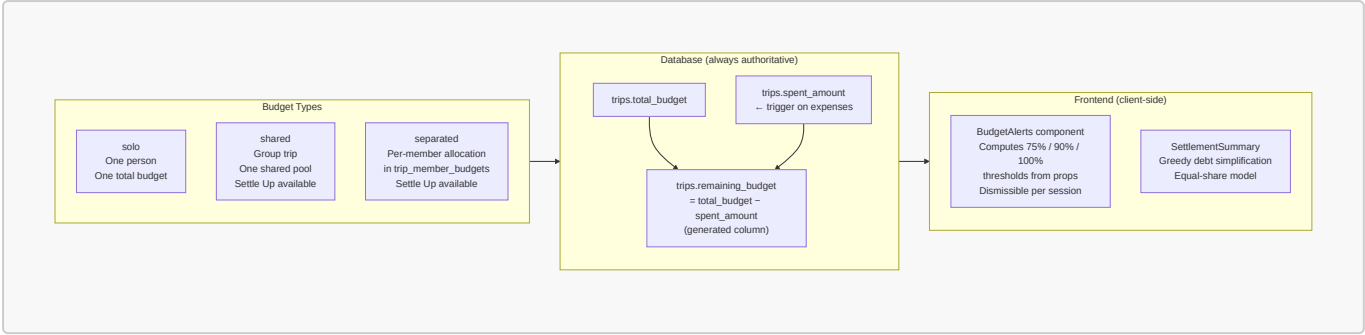
Trip Flow



Expense Flow



Budget System



API Endpoints Reference

Auth — /api/auth

Method	Path	Body	Response
POST	/initiate-signup	{ email }	200 + message
POST	/verify-email	{ email, code, purpose }	{ verificationToken }
POST	/complete-signup	{ verificationToken, name, password }	LoginResponse
POST	/login	{ email, password }	LoginResponse
POST	/forgot-password	{ email }	200 + message
POST	/reset-password	{ verificationToken, newPassword }	200 + message
POST	/refresh	{ refreshToken }	LoginResponse
GET	/health	—	200

LoginResponse:{ accessToken, refreshToken, expiresIn, user: { id, name, email, avatar, authProvider, country, currency, onboardingCompleted } }

Trips — /api/trips (auth required)

Method	Path	Notes
GET	/	All non-archived trips for current user
POST	/	Create trip
GET	/ {tripId}	Trip detail — owner or member only
PUT	/ {tripId}	Update trip — owner or editor only
DELETE	/ {tripId}	Delete trip — owner only; cascades to all child tables

Expenses — </api/expenses> (*auth required*)

Method	Path	Notes
POST	/	Add expense; include subItems[] for bundle category
GET	/trip/{tripId}	All expenses for trip, newest first; includes subItems[]
DELETE	/{expenseId}	Delete — expense creator or trip owner only

Itinerary — </api/trips/{tripId}/itinerary> (*auth required*)

Method	Path	Notes
GET	/	All items ordered by date + start_time
POST	/	Add item
PUT	/{itemId}	Update item
DELETE	/{itemId}	Delete item

Packing — </api/trips/{tripId}/packing> (*auth required*)

Method	Path	Notes
GET	/	All packing items
POST	/	Add item
PATCH	/{itemId}/toggle	Toggle is_checked
DELETE	/{itemId}	Delete item
POST	/preset?type=	Add preset (general , beach , hiking , business , umrah)

Activity — </api/trips/{tripId}/activity> (*auth required*)

Method	Path	Notes
GET	/	Activity feed, newest first

Settlement — </api/trips/{tripId}/settlement> (*auth required*)

Method	Path	Notes
GET	/	Calculate split: balances + who-owes-whom transactions

Users — </api/users> (*auth required*)

Method	Path	Notes
GET	/me	Current user profile

Method	Path	Notes
PUT	<code>/profile</code>	Update name, country, currency
PUT	<code>/change-password</code>	Change password (requires current password)

Frontend Pages & Components

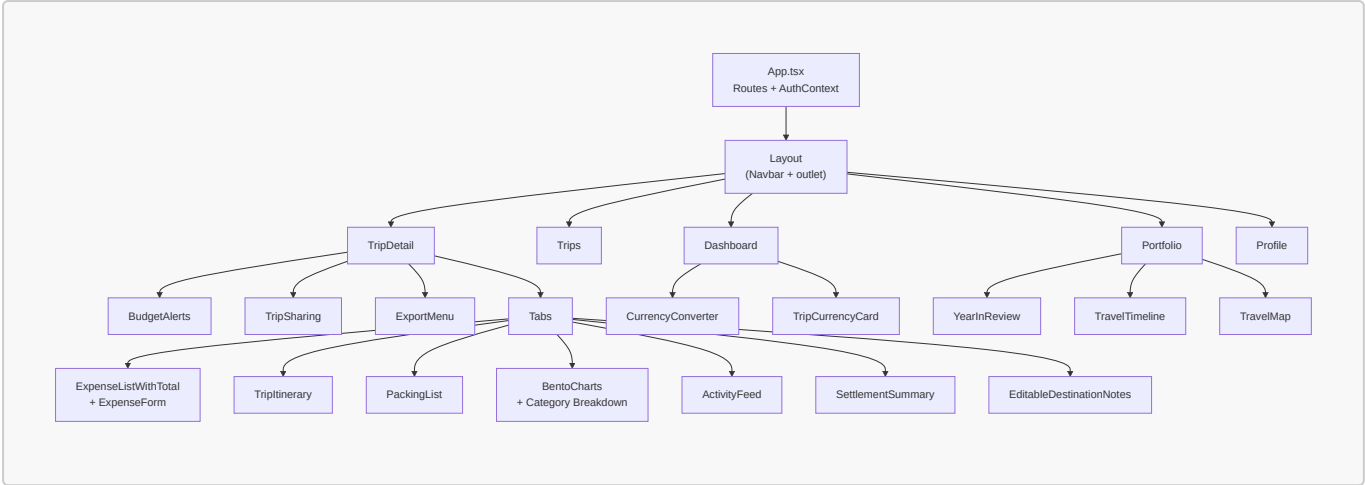
Pages

Page	Route	What it does
Landing	<code>/</code>	Marketing page; auto-redirects to dashboard if logged in
Login	<code>/login</code>	Email/password + Google login
Register	<code>/register</code>	3-step email signup flow
ForgotPassword	<code>/forgot-password</code>	OTP-based password reset
OAuthCallback	<code>/oauth2/callback</code>	Receives tokens from Google redirect
Dashboard	<code>/dashboard</code>	Overview stats, upcoming trips, Quick Price Converter
Trips	<code>/trips</code>	Trip list with filters, create/edit/delete/archive
TripDetail	<code>/trips/:id</code>	Full trip view with all feature tabs
Portfolio	<code>/portfolio</code>	Year in review, travel timeline, travel map
Profile	<code>/profile</code>	Account settings, password change
NotFound	<code>*</code>	404 page

TripDetail Tabs

Tab	Component	Visibility
Expenses	<code>ExpenseListWithTotal</code>	Always
Itinerary	<code>TripItinerary</code>	Always
Packing	<code>PackingList</code>	Always
Charts	<code>BentoCharts</code> + category breakdown	Always
Activity	<code>ActivityFeed</code>	Always
Settle Up	<code>SettlementSummary</code>	Only for <code>shared</code> and <code>separated</code> trips
Notes	<code>EditableDestinationNotes</code>	Always

Component Map



Auth State in localStorage

Key	Value
travelluhh_token	Access JWT (24h)
travelluhh_refresh_token	Refresh JWT (7d)
travelluhh_user	Serialized user object
travelluhh_notes_<tripId>	Per-trip destination notes array
travelluhh_destinfo_<tripId>	Per-trip destination quick info

Known State / Tech Debt

No Soft Delete (by design)

All tables use hard deletes. `is_deleted` columns have been dropped throughout. Intentional — personal-use app, no audit log requirement.

Edit Expense Not Supported

`handleUpdateExpense` in `TripDetail.tsx` shows a toast: *"Edit not supported yet."* Workaround: delete and re-add. To fix: implement `PUT /api/expenses/{id}` and wire `useExpenses.updateExpense`.

Solo Budget Type Not Tested End-to-End

Trip creation with `budget_type = solo` exists but full flow (create → add expenses → view charts) has not been manually verified.

Shared / Separated Budget With Multiple Users Not Tested

The settlement and per-member budget flows have not been tested with real multi-user scenarios. Only the ownerself has been used in testing.

Trip Sharing Is UI-Only

`TripSharing` dialog renders correctly with real member data, but:

- **Invite by email** — `onAddCollaborator` callback is not wired to any API call
- **Remove member** — `onRemoveCollaborator` callback is not wired
- **Change role** — `onUpdateCollaboratorRole` callback is not wired

No backend endpoints for member management exist. Only the initial auto-add of the trip creator as `owner` works.

Destination Notes Are localStorage-Only

The `destination_notes` DB table exists but is unused. Notes are stored client-side in `localStorage` keyed by `travelluhh_notes_<tripId>`. Data is device/browser-local and will be lost if `localStorage` is cleared.

Budget Alerts Are Computed Client-Side

The `budget_alerts` DB table exists but is unused. Alerts are computed in `BudgetAlerts.tsx` from props at 75%, 90%, 100% thresholds. Dismissal resets on page refresh.

Settlement Is Equal-Share Only

Settlement algorithm divides total trip spend equally by member count. Does not account for unequal allocations or who actually paid what. For `separated` mode, consider switching to a per-allocation model.

Activity Log Coverage Is Partial

Only expense add/delete and trip update actions are currently logged to `trip_activities`. Packing, itinerary changes, and member role changes are not logged.

Refresh Tokens Not Stored in DB

`refresh_tokens` table exists in schema but is unused. Refresh token validation is done by JWT signature only, not by DB lookup. Tokens cannot be revoked (e.g. on logout) — clearing `localStorage` is the only logout mechanism.

CORS

Currently configured for `localhost:5173`, `localhost:8081`, `localhost:8082`. Update `application.yml` when deploying.

Not Yet Built

These features have partial scaffolding (UI, DB table, or type definitions) but are not functional end-to-end:

Feature	Status	What's Missing
Trip member invite	UI-only	Backend endpoint + wiring in TripDetail
Remove / change role of member	UI-only	Backend endpoints + wiring

Feature	Status	What's Missing
Separated budget allocation	DB table + UI component exist (<code>BudgetAllocationPanel</code>) but not mounted anywhere	API endpoints + wiring into TripDetail
Edit expense	UI toast says "not supported"	<code>PUT /api/expenses/{id}</code> + <code>useExpenses.updateExpense</code>
Destination notes backend	<code>destination_notes</code> table in DB	API CRUD + replace localStorage with real persistence
Budget alerts persistence	<code>budget_alerts</code> table in DB	API + store/fetch alert dismissals from DB
Refresh token revocation	<code>refresh_tokens</code> table in DB	Store on issue, check on refresh, revoke on logout
Real-time collaboration	—	WebSocket or SSE for live multi-user updates
Push / email notifications	—	Alert emails when budget thresholds are crossed
Trip image upload	Image field exists, <code>image</code> URL stored	File upload endpoint (S3 or local storage)
Deep linking / public trip view	Public access toggle in TripSharing	Backend auth bypass for public trips